

ENGENHARIA DE COMPUTAÇÃO

Disciplina: Engenharia de Software

DOCUMENTO DE ARQUITETURA DE SOFTWARE (SAD)

KA 02 – SWEBOK v4.0

SisConsult

Sistema de Gestão de Consultório Médico

Versão: 2.0

Data: Junho de 2026

Instituição: Universidade do Oeste de Santa Catarina - UNOESC

Professor: Odemir Moreira da Mata Jr.

Equipe: Arthur H. Menegat, Arthur Kuznier, Gabriel D. Besbati, Giacomo Bortolon,
Lucas Locatelli, Luiz Mozzer

Histórico de Revisões

Versão	Data	Autor(es)	Descrição da Alteração	Revisado por
1.0	Fevereiro/2026	Equipe	Criação inicial do documento de Ar-quitetura (KA02)	Grupo
2.0	Junho/2026	Equipe	Revisão (KA02)	Equipe

Conteúdo

Histórico de Revisões	1
1 Documento de Arquitetura de Software (SAD)	3
1.1 Estilo Arquitetural Escolhido e Justificativa	3
1.2 Atributos de Qualidade Priorizados	3
1.3 Visões Arquiteturais	3
2 Registros de Decisões Arquiteturais (ADRs)	4
3 Diagramas Arquiteturais (C4 Model)	6
3.1 Diagrama de Contexto (Nível 1)	6
3.2 Diagrama de Contêineres (Nível 2)	6

1 Documento de Arquitetura de Software (SAD)

1.1 Estilo Arquitetural Escolhido e Justificativa

O estilo arquitetural escolhido é o **Monolito Modular** baseado no modelo Client-Server (Frontend separado do Backend).

- **Justificativa:** Considerando que a aplicação web visa atender um consultório médico de pequeno e médio porte, uma arquitetura de microsserviços introduziria uma complexidade operacional e de infraestrutura desnecessária nesta fase. O Monolito Modular garante uma implantação simplificada e, ao mesmo tempo, impõe limites lógicos rigorosos (módulos) no nível do código, o que facilitará uma futura extração de serviços caso o consultório escale. Além disso, a separação clara entre as camadas de apresentação, negócio e dados ajuda a isolar as regras de acesso a informações sensíveis.

1.2 Atributos de Qualidade Priorizados

A arquitetura foi projetada para suportar os seguintes Requisitos Não Funcionais (RNFs):

- **Segurança e Privacidade:** O sistema lida com prontuários, históricos clínicos e diagnósticos, sendo obrigatório o suporte arquitetural aos ditames da Lei Geral de Proteção de Dados - LGPD. A arquitetura garante isso por meio da segregação de dados em esquemas de banco de dados específicos e validação rigorosa de autorização via RBAC (Role-Based Access Control).
- **Disponibilidade:** A arquitetura prevê redundância no nível do servidor de aplicação e banco de dados para garantir que médicos e recepcionistas tenham acesso ininterrupto à agenda e aos prontuários durante o expediente.
- **Manutenibilidade:** O particionamento em módulos lógicos (Autenticação, Pacientes, Prontuário, etc.) reduz o acoplamento, permitindo que a evolução de um módulo (ex: adição de novos relatórios no Dashboard) não impacte as regras de negócio de outro.

1.3 Visões Arquiteturais

- **Visão de Contexto:** O sistema interage primariamente com três perfis de usuários humanos: o Médico, a Recepcionista e o Administrador. A aplicação web encapsula todo o fluxo de trabalho interno do consultório, sem dependências críticas externas de tempo real nesta primeira versão (com exceção de serviços básicos como envio de e-mails/SMS para pacientes).
- **Visão de Contêineres:** O sistema é dividido em três contêineres principais: uma Aplicação Web Single Page Application (SPA) no frontend, uma API Backend (que centraliza a lógica de negócios e as regras de autorização) e um Banco de Dados Relacional.
- **Visão de Componentes:** A API Backend divide-se internamente nos componentes de negócio exigidos: Gestão de Identidade/Perfis, Gestão de Pacientes, Agendamento, Prontuário Eletrônico, Prescrições, Exames e Geração de Relatórios/Dashboard.

2 Registros de Decisões Arquiteturais (ADRs)

Abaixo estão formalizadas as principais decisões arquiteturais adotadas durante a fase de design, documentando o contexto, as alternativas consideradas e suas consequências diretas no projeto.

ADR 001: Adoção do Estilo Monolito Modular	
Contexto:	O sistema gerenciará as rotinas de um consultório de pequeno e médio porte, exigindo integração fluida entre as áreas de agendamento e atendimento médico.
Decisão:	Adotar a arquitetura de Monolito Modular, com separação apenas entre Frontend e Backend.
Alternativas:	Arquitetura de Microserviços; Monolito Tradicional (com visualizações renderizadas diretamente no servidor).
Consequências:	Menor custo de infraestrutura inicial e facilidade de deploy. Exige disciplina estrita da equipe de desenvolvimento para não burlar os limites lógicos entre os módulos do código-fonte.
ADR 002: Proteção e Privacidade dos Dados (LGPD)	
Contexto:	O sistema manipula dados pessoais sensíveis de saúde (prontuários, diagnósticos e alergias), com obrigações rigorosas impostas pela legislação de privacidade.
Decisão:	Os dados sensíveis do prontuário serão fisicamente ou logicamente segregados das tabelas de identificação básica do paciente. Adicionalmente, toda requisição de acesso ao prontuário deverá passar por um interceptor arquitetural que registrará um log de auditoria inalterável.
Alternativas:	Armazenar todos os dados em uma única estrutura sem logs compulsórios; Criptografia no nível da aplicação para todos os campos (descartada devido ao impacto na performance de consultas e relatórios).
Consequências:	Aumenta a complexidade de gravação e leitura no banco de dados, mas garante as trilhas de auditoria essenciais para conformidade legal.

ADR 003: Controle de Acesso Baseado em Perfis (RBAC)	
Contexto:	A aplicação possui perfis distintos de usuários, notadamente: médico, recepcionista e administrador, cada um com permissões rigorosamente separadas na regra de negócio.
Decisão:	Implementar controle de acesso na camada de API utilizando Role-Based Access Control (RBAC), atrelado a tokens JWT (JSON Web Tokens).
Alternativas:	Attribute-Based Access Control (ABAC), que oferece maior granularidade, porém foi considerado excessivamente complexo para os 3 perfis iniciais descritos.
Consequências:	Validação centralizada, segura e rápida de permissões em cada requisição à API. Em caso de criação de subperfis no futuro, o modelo precisará ser estendido.
ADR 004: Comunicação Frontend e Backend via API RESTful	
Contexto:	Necessidade de prover uma interface rápida e ágil para uso contínuo durante as consultas médicas e no atendimento presencial da recepção.
Decisão:	A comunicação entre o Frontend Web e a camada de serviços (Backend) será feita exclusivamente via uma API RESTful operando sobre o protocolo HTTPS/TLS.
Alternativas:	GraphQL (descartado pois as entidades e os relacionamentos são diretos na fase inicial); gRPC (melhor voltado para tráfego entre serviços internos, e não para browsers web padrão).
Consequências:	Padroniza a comunicação em um formato amplamente conhecido e facilita a integração futura de novos clientes, como um possível aplicativo mobile para o paciente.
ADR 005: Armazenamento de Arquivos e Prescrições	
Contexto:	O sistema precisará gerenciar receituários digitais e solicitações de exames, que frequentemente geram ou exigem upload de documentos no formato PDF.
Decisão:	Armazenar os arquivos e documentos não estruturados em um Object Storage (ex: AWS S3 ou equivalente compatível) separado do Banco de Dados Relacional, guardando no banco de dados apenas o referencial seguro (URI) do arquivo.
Alternativas:	Salvar os PDFs diretamente no banco de dados relacional como campos do tipo BLOB.
Consequências:	O banco de dados relacional permanecerá leve, rápido e performático. Contudo, introduz-se um segundo sistema de armazenamento que deverá, obrigatoriamente, herdar as mesmas políticas de segurança (LGPD) e rotinas de backup da base principal.

3 Diagramas Arquiteturais (C4 Model)

As representações visuais abaixo detalham a arquitetura estática do Sistema de Gestão de Consultório Médico, orientando o entendimento de sua composição desde o contexto externo até a estruturação interna dos contêineres.

3.1 Diagrama de Contexto (Nível 1)

Este diagrama ilustra como o Sistema de Gestão se insere no ambiente do consultório médico e as interações com seus principais atores.

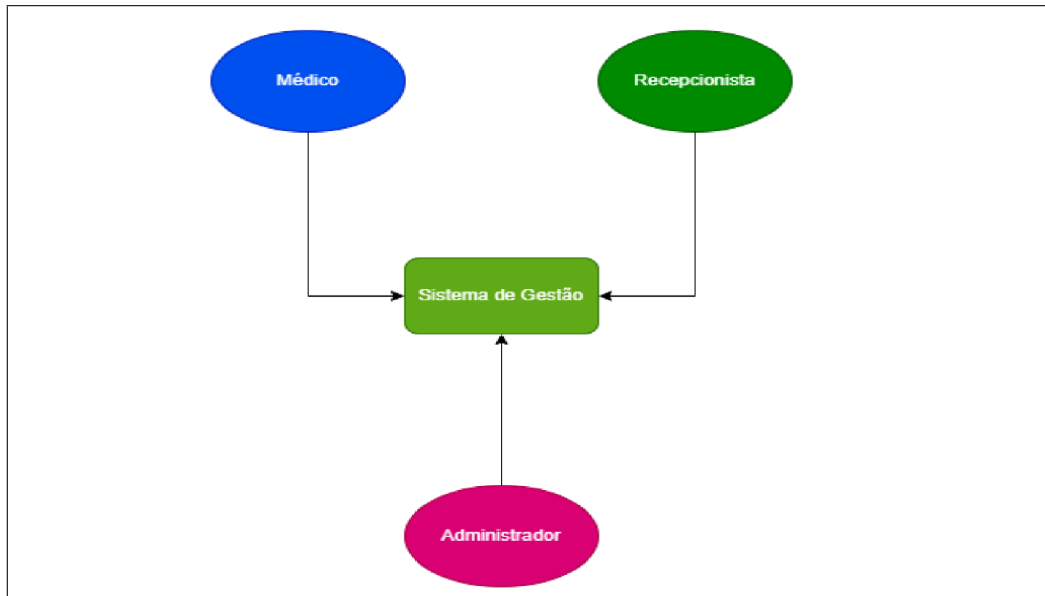


Figura 1: Diagrama de Contexto (Nível 1)

3.2 Diagrama de Contêineres (Nível 2)

Este diagrama aprofunda a visão de arquitetura e detalha a decomposição interna do sistema nos contêineres essenciais que sustentam a operação.

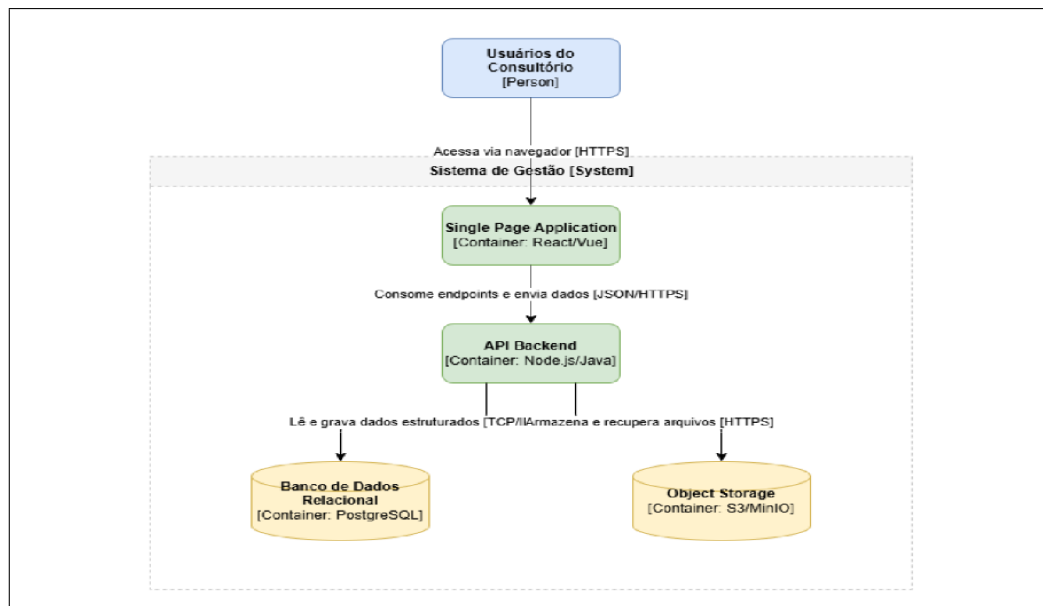


Figura 2: Diagrama de Contêineres (Nível 2)